



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Δ.Π.Μ.Σ. «Επιστήμη Δεδομένων και Μηχανική Μάθηση»

Διαχείριση Δεδομένων Μεγάλης Κλίμακας

Εξαμηνιαία εργασία

Φοιτητής Γεώργιος Σκουρτσίδης
A.M. 03400314

Διδάσκοντες Δημήτριος Τσουμάκος
Ιωάννης Κωνσταντίνου

Υπεύθυνος εργαστηρίου Νικόλαος Χαλβατζής

Ακαδημαϊκό έτος 2025-2026

1 Περιβάλλον και εκτέλεση

Η εργασία υλοποιήθηκε σε Java 17 και εκτελείται πάνω σε Spark 3.5.8 και Hadoop 3.4.1, στο Kubernetes cluster του εργαστηρίου. Η υποβολή γίνεται με χρήση ενός βοηθητικού script (submit.sh) σε cluster mode. Ο driver και οι executors δημιουργούνται ως εφήμερα pods από την εικόνα apache/spark:3.5.8-java17.

Η προεπιλεγμένη εικόνα του Spark χρησιμοποιεί Java 11, οπότε δηλώνεται ρητά το αντίστοιχο java17 tag.

Κάθε query χωρίζεται σε μια συνάρτηση compute() με τη λογική και ένα μικρό main που φορτώνει τα δεδομένα, χρονομετρά και τυπώνει. Αυτό γίνεται για λόγους testability: με χρήση τοπικών unit tests, ελέγχουμε την ορθότητα χωρίς καν να χρειάζεται cluster.

Αποθετήριο κώδικα (code repository) <https://github.com/SkourtsidisGiorgos/ntua-bigdata-2026>

2 Ανάλυση δεδομένων

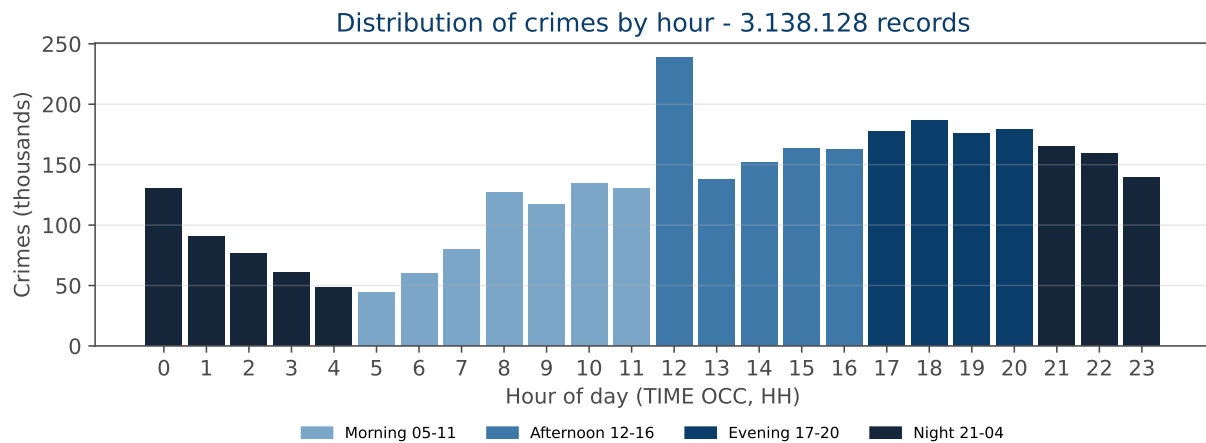
Πριν την υλοποίηση έγινε μια πρώτη εξέταση των δεδομένων (EDA) με χρήση jupyter notebook. Τα δεδομένα αποθηκεύτηκαν τοπικά από το remote HDFS και στη συνέχεια διαβάστηκαν με pandas και geopandas.

Παρατηρήσεις:

- **Crime data** (609 MB και 288 MB, 28 στήλες, CSV με εισαγωγικά): τα δύο αρχεία ενώνονται. Η ημερομηνία δίνεται σε μορφή 2010 Feb 20 12:00:00 AM και η ώρα ως ακέραιος HHMM. Η τιμή STREET ανήκει στη στήλη Premis Desc. Στις συντεταγμένες υπάρχουν αρκετές εγγραφές (0,0), που αντιστοιχούν σε άγνωστη θέση και απορρίπτονται στο Query 4.
- **Census blocks** (GeoJSON, ~172 MB): ένα Feature ανά γραμμή. Με multiLine=true ολόκληρο το αρχείο διαβαζόταν ως ένα record και προκαλούσε OOM στους executors. Επιλέχθηκε ανάγνωση γραμμή προς γραμμή, κατανεμημένη και χαμηλή σε μνήμη. Διατηρούνται ZCTA20, POP20, HOUSING20.
- **Income** (CSV με «;», 282 γραμμές): το εισόδημα δίνεται σε μορφή «\$52,806», τα σύμβολα αφαιρούνται και η τιμή μετατρέπεται σε ακέραιο. Δύο zip δεν έχουν εισόδημα.
- **Police stations** (21 γραμμές): η κεφαλίδα φέρει BOM, οπότε οι στήλες λαμβάνονται κατά θέση. Το x αντιστοιχεί στο γεωγραφικό μήκος και το y στο πλάτος.

Πέρα από την EDA στο notebook, γράφτηκαν tests που διαβάζουν τα πραγματικά αρχεία και επαληθεύουν τα μεγέθη (282 γραμμές με 2 κενές τιμές, 21 σταθμοί, περίπου 300 zip).

Ακολουθεί ένα παράδειγμα από το EDA: η κατανομή των εγκλημάτων ανά ώρα της ημέρας (Σχήμα 1). Η εγκληματικότητα είναι χαμηλότερη τις πρώτες πρωινές ώρες (04:00–05:00) και αυξάνεται προς το βράδυ.



Σχήμα 1: Κατανομή των εγκλημάτων ανά ώρα, χρωματισμένη κατά τμήμα ημέρας (όρια Query 1). Η κορυφή στις 12:00 είναι artifact, όπου οι άγνωστες ή μη καταγεγραμμένες ώρες αποθηκεύονται ως μεσημέρι

3 Μορφή αποθήκευσης

Τα δεδομένα δίνονται σε CSV. Τα CSV είναι αρχεία κειμένου χωρίς σχήμα, δεν έχει συμπίεση και είναι row oriented, οπότε το Spark διαβάζει και να αναλύει όλες τις στήλες κάθε γραμμής.

Υπάρχουν καταλληλότερες μορφές για αναλυτικά φορτία. Το Parquet και το ORC είναι columnar, δηλαδή αποθηκεύουν τα δεδομένα κατά στήλη, οπότε ένα ερώτημα διαβάζει μόνο όσες στήλες χρειάζεται (column pruning). Η αποθήκευση κατά στήλη δίνει και καλύτερη συμπίεση, καθώς οι τιμές κάθε στήλης είναι ομοιογενείς, ενώ τα στατιστικά ανά block (max, min, count) επιτρέπουν να παρακάμπτονται ολόκληρα blocks χωρίς ανάγνωση (predicate pushdown). Το Avro είναι αντίθετα row based, διαθέτει όμως σχήμα και δυαδική, συμπαγή αναπαράσταση, και ταιριάζει καλύτερα σε σενάρια εγγραφής ή ροής με εξέλιξη σχήματος (schema evolution). Τα Parquet και ORC είναι παρόμοια στην λειτουργία τους. Επιλέχθηκε το Parquet, καθώς αποτελεί την προεπιλεγμένη columnar μορφή στο Spark, και έχει καλή συμπίεση και υψηλή απόδοση.¹

Για τη σύγκριση, τα δεδομένα crime μετατράπηκαν σε Parquet και το ίδιο Query 2 εκτελέστηκε και στις δύο μορφές, με 4 executors.

Μορφή (Query 2, 4×1c/2GB)	Χρόνος (ms)	Μέγεθος στο HDFS
CSV	57.825	~897 MB
Parquet	6.524	140 MB

Πίνακας 1: Επίδραση της μορφής αποθήκευσης. Η εφάπαξ μετατροπή κόστισε 89.787 ms.

Η διαφορά είναι περίπου εννεαπλάσια, με ταυτόχρονη μείωση του μεγέθους κατά έξι φορές. Το Query 2 χρειάζεται μόνο τις στήλες year και month, οπότε από τις 28 στήλες οι 26 αγνοούνται. Αυτό εξηγεί γιατί το Parquet είναι ~140 MB έναντι 897 MB του CSV και ο χρόνος έπεσε από 57.825 σε 6.524 ms. Το κέρδος προέρχεται

¹Πηγές: τεκμηρίωση Apache Parquet (<https://parquet.apache.org/docs/>) και Apache Spark SQL: Data Sources (<https://spark.apache.org/docs/latest/sql-data-sources.html>), επίσης B. Chambers & M. Zaharia, *Spark: The Definitive Guide*, O'Reilly, 2018, κεφ. 9.

και από το column pruning και από την απουσία του κόστους parsing του CSV. Η μετατροπή έχει κόστος περίπου 90 δευτερόλεπτα.

4 Query 1 (DataFrame, UDF, RDD)

Ζητείται η κατάταξη των τμημάτων της ημέρας με βάση το ποσοστό των εγκλημάτων που σημειώθηκαν στον δρόμο. Τα όρια είναι: Πρωί 05:00 ως 11:59, Απόγευμα 12:00 ως 16:59, Βράδυ 17:00 ως 20:59, Νύχτα 21:00 ως 04:59. Ο παρονομαστής (σύνολο εγγραφών) είναι κοινός για όλα τα τμήματα, επομένως δεν επηρεάζει την κατάταξη. Το ποσοστό δίνεται επί του συνόλου των εγγραφών με έγκυρη ώρα (3.138.128 εγγραφές). Και οι τρεις υλοποιήσεις παράγουν παρεμφερή αποτέλεσμα:

Τμήμα ημέρας	Εγκλήματα STREET	Ποσοστό (%)
Νύχτα (Night)	251.094	8,0014
Βράδυ (Evening)	198.292	6,3188
Απόγευμα (Afternoon)	156.432	4,9849
Πρωί (Morning)	130.866	4,1702

Πίνακας 2: Query 1: ποσοστό εγκλημάτων STREET ανά τμήμα ημέρας.

Υλοποίηση (2 exec × 1c/2GB)	Χρόνος (ms)
DataFrame	46.966
DataFrame + UDF	43.163
RDD	28.055

Πίνακας 3: Query 1: χρόνοι εκτέλεσης.

Περιμέναμε να υπερτερεί το DataFrame. Αντ' αυτού, η RDD υλοποίηση ήταν ταχύτερη. Το αποτέλεσμα πιθανόν οφείλεται στο ότι το query είναι ελαφρύ. Ένα φίλτράρισμα και μια συγκέντρωση σε τέσσερα κλειδιά, χωρίς βαρύ shuffle. Η RDD διαβάσει μόνο δύο στήλες και εκτελεί reduceByKey με τοπική σύνθεση (map-side combine), χωρίς το επιπλέον κόστος του πλάνου. Το UDF παρουσίασε ουσιαστικά τον ίδιο χρόνο με το καθαρό DataFrame. Το UDF αποτελεί black box για τον Catalyst και δεν συμμετέχει στο whole-stage codegen, αλλά εδώ η κατηγοριοποίηση της ώρας είναι αρκετά φθηνή ώστε η διαφορά να είναι αμελητέα.

5 Query 2 (DataFrame και SQL)

Για κάθε έτος ζητούνται οι 3 μήνες με τα περισσότερα εγκλήματα και η θέση τους στην κατάταξη. Χρησιμοποιείται ROW_NUMBER σε παράθυρο ανά έτος, με tie-break στον μήνα ώστε το top-3 να είναι ντετερμινιστικό. Οι δύο υλοποιήσεις δίνουν ταυτόσημο αποτέλεσμα, 48 γραμμές. Για το 2010, για παράδειγμα, προκύπτουν οι μήνες 1, 3 και 7.

Η διαφορά στους χρόνους δεν θεωρείται σημαντική. Και οι δύο διατυπώσεις περνούν από τον ίδιο Catalyst και καταλήγουν στο ίδιο φυσικό πλάνο, οπότε αναμένονται παρόμοιοι χρόνοι. Το μεγαλύτερο κόστος είναι το διάβασμα του CSV και το shuffle της GROUP BY. Έγιναν πολλά τρεξίματα που έδωσαν διαφορετικούς χρόνους, γεγονός που αποδίδεται στον φόρτο του κοινόχρηστου cluster. Το τελευταίο

Υλοποίηση (4 exec × 1c/2GB)	Χρόνος (ms)
DataFrame	57.825
SQL	35.234

Πίνακας 4: Query 2: χρόνοι εκτέλεσης.

ενισχύεται από το γεγονός ότι τα πειράματα έτρεξαν την τελευταία ημέρα της προθεσμίας και προσωπική εκτίμηση είναι ότι πολλοί φοιτητές έτρεξαν τα queries τους την ίδια ημέρα.

6 Query 3 (DataFrame και RDD)

Ζητείται το μέσο κατά κεφαλήν εισόδημα ανά zip για την περίοδο 2020-2021. Το διαθέσιμο εισόδημα που μας δίνεται το dataset είναι ανά νοικοκυριό και όχι ανά άτομο. Έγινε μετατροπή σε κατά κεφαλήν, διαιρώντας με το μέσο μέγεθος νοικοκυριού, το οποίο προκύπτει από το census:

$$\text{κατά κεφαλήν} = \frac{\text{εισόδημα νοικοκυριού}}{\text{πληθυσμός/νοικοκυριά}} = \text{εισόδημα} \cdot \frac{\text{νοικοκυριά}}{\text{πληθυσμός}}.$$

Τα νοικοκυριά προσεγγίζονται από το HOUSING20 και το join γίνεται στο zip. Οι δύο υλοποιήσεις συμφωνούν (280 zip). Στις υψηλότερες θέσεις εμφανίζονται περιοχές όπως Malibu, Playa Vista και Pacific Palisades. Με αναζήτηση στο google επιβεβαιώθηκε πως πρόκειται για ακριβές περιοχές, κάτι που αποτελεί ένδειξη ορθότητας του υπολογισμού.

Υλοποίηση (3 exec × 1c/2GB)	Χρόνος (ms)
DataFrame (αυτόματα BroadcastHashJoin)	9.885
RDD (join = hash join)	17.208

Πίνακας 5: Query 3: χρόνοι εκτέλεσης.

Εδώ το DataFrame είναι ξεκάθαρα καλύτερο, αντίστροφα από το Query 1. Ο Catalyst αναγνωρίζει ότι η πλευρά του εισοδήματος είναι μικρή και επιλέγει BroadcastHashJoin, την εκπέμπει δηλαδή χωρίς shuffle. Η RDD join δεν διαθέτει αντίστοιχη βελτιστοποίηση και επιβάλλει shuffle και στις δύο πλευρές. Όπου υπάρχει join, το DataFrame είναι προτιμότερο, ενώ όπου πρόκειται για απλή συγκέντρωση, η RDD επαρκεί.

7 Query 4 και scalability

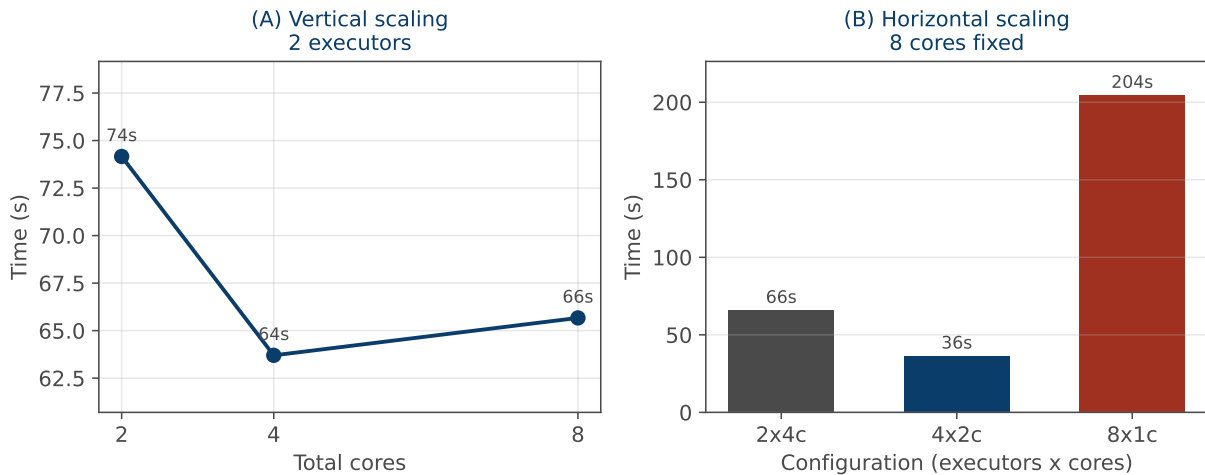
Για κάθε σταθμό ζητείται το πλήθος των εγκλημάτων που βρίσκονται πλησιέστερα σε αυτόν παρά σε οποιονδήποτε άλλον, καθώς και η μέση απόσταση. Πρόκειται ουσιαστικά για πρόβλημα πλησιέστερου γείτονα: κάθε έγκλημα συγκρίνεται με τους 21 σταθμούς. Η υλοποίηση χρησιμοποιεί cross join. Επειδή η πλευρά των σταθμών είναι ελάχιστη, ο Spark επιλέγει BroadcastNestedLoopJoin και τα εγκλήματα δεν μετακινούνται για το join. Στη συνέχεια, ανά έγκλημα κρατείται ο κοντινότερος σταθμός μέσω min πάνω σε struct(απόσταση, σταθμός) και ακολουθεί συγκέντρωση

ανά σταθμό. Η απόσταση haversine υλοποιήθηκε ως έκφραση στήλης και όχι ως UDF, ώστε να παραμένει εντός του codegen, καθώς αποτιμάται περίπου 63 εκατομμύρια φορές. Πρώτο εμφανίζεται το HOLLYWOOD με 225.515 εγκλήματα και μέση απόσταση 2,077 km.

Κατά την κλιμάκωση εμφανίστηκε πρόβλημα με τη μνήμη. Στη μικρότερη διαμόρφωση (1 πυρήνας, 2 GB) η εκτέλεση ολοκληρωνόταν κανονικά, ενώ με 2 πυρήνες ανά executor οι executors τερματίζονταν με OOM. Το πρόβλημα αυτό αρχικά φαινόταν ανορθόδοξο, καθώς η διαθέσιμη μνήμη ήταν περισσότερη. Η αιτία εντοπίστηκε στην *off-heap* μνήμη. Με περισσότερους πυρήνες εκτελούνται ταυτόχρονα tasks που γεμίζουν τα buffers του shuffle πάνω από το προεπιλεγμένο memoryOverhead, οπότε το Kubernetes τερματίζει το pod. Το πρόβλημα αντιμετωπίστηκε με αύξηση του memoryOverhead, που είναι ανεξάρτητο από τη heap μνήμη της εκφώνησης. Σημειώνεται επίσης ότι η πρώτη εκτέλεση κάθε σειράς ήταν πιο αργή λόγω cold start και ότι, λόγω του κοινόχρηστου cluster, οι μετρήσεις έδιναν διαφορετικούς χρόνους.

	Διαμόρφωση	Σύν. cores	Χρόνος (ms)
A	2 exec × 1c / 2GB	2	74.162
	2 exec × 2c / 4GB	4	63.700
	2 exec × 4c / 8GB	8	65.669
B	2 exec × 4c / 8GB	8	65.669
	4 exec × 2c / 4GB	8	36.223
	8 exec × 1c / 2GB	8	204.331

Πίνακας 6: Query 4: κλιμακωσιμότητα. Πάνω κάθετη (A), κάτω οριζόντια (B).



Σχήμα 2: Query 4: κλιμακωσιμότητα. (A) Κάθετη: από 4 πυρήνες και μετά ο χρόνος σταθεροποιείται (shuffle-bound). (B) Οριζόντια με σταθερά 8 cores: η ενδιάμεση διαμόρφωση 4×2c είναι ταχύτερη, ενώ οι 8 μικροί executors του ενός πυρήνα είναι ο χειρότερος συνδυασμός.

Στην κάθετη κλιμάκωση (A), η μετάβαση από 2 σε 4 πυρήνες μείωσε τον χρόνο, ενώ από 4 σε 8 η βελτίωση ήταν αμελητέα. Το ερώτημα δεν περιορίζεται από τη CPU αλλά από το shuffle: το groupBy ανά έγκλημα είναι το ακριβό στάδιο. Από ένα σημείο και ύστερα οι επιπλέον πυρήνες παραμένουν αναξιοποίητοι. Στην οριζόντια κλιμάκωση (B), με σταθερά τα 8 cores, ταχύτερη ήταν η διαμόρφωση 4 executors των 2 πυρήνων. Το τρέξιμο με 2 executors των 4 πυρήνων ήταν πιο αργό, ενώ με 8 executors του ενός πυρήνα ήταν ο χειρότερος συνδυασμός, με 204 δευτερόλεπτα. Πολλοί μικροί executors συνεπάγονται περισσότερο shuffle στο δίκτυο και λιγότερη μνήμη

ανά executor, άρα περισσότερο spill. Συνολικά, μια ενδιαμέση διαμόρφωση με λίγους executors των δύο πυρήνων αποδίδει καλύτερα από τα δύο άκρα.

8 Στρατηγικές join

Με χρήση των hint και explain εξετάστηκαν οι στρατηγικές join που επιλέγει ο Catalyst.

Στο Query 3 υπάρχει κλειδί ισότητας (το zip), οπότε ισχύουν και οι τρεις βασικές στρατηγικές. Με κάθε hint, το explain επιβεβαιώνει τον αντίστοιχο τελεστή:

Hint	Φυσικό join (από explain)	Χρόνος (ms)
BROADCAST	BroadcastHashJoin	13.516
MERGE	SortMergeJoin	21.245
SHUFFLE_HASH	ShuffledHashJoin	16.172
SHUFFLE_REPLICATE_NL	CartesianProduct	12.814

Πίνακας 7: Query 3: επίδραση στρατηγικής join (3 exec $\times$ 1c/2GB).

Οι τρεις από τις τέσσερις στρατηγικές δεν διαφέρουν σημαντικά, με εξαίρεση το SORT-MERGE. Πριν το join, το census έχει ήδη συγκεντρωθεί ανά zip, από 91 χιλιάδες blocks σε περίπου 300 γραμμές. Συνεπώς και οι δύο πλευρές του join είναι πλέον μικρές και το πραγματικό κόστος βρίσκεται αλλού, στο διάβασμα του GeoJSON. Το SORT-MERGE ήταν παρ' όλα αυτά σταθερά το πιο αργό, καθώς ταξινομεί χωρίς λόγο και τις δύο πλευρές. Ο αυτόματος χρόνος του Πίνακα 5, 9.885 ms, προέρχεται από προγενέστερη εκτέλεση, γι' αυτό το ρητό BROADCAST εδώ μετρήθηκε ελαφρώς υψηλότερα, πρόκειται για διακύμανση.

Στο Query 4 δεν υπάρχει κλειδί ισότητας, πρόκειται για cross join, οπότε νόημα έχουν μόνο οι nested-loop στρατηγικές. Αυτό επιβεβαιώθηκε και πειραματικά: με hint MERGE, ο Catalyst το αγνόησε και επέστρεψε σε BroadcastNestedLoopJoin.

Hint	Φυσικό join (από explain)	Χρόνος (ms)
BROADCAST	BroadcastNestedLoopJoin, Cross	74.162
SHUFFLE_REPLICATE_NL	CartesianProduct	70.346

Πίνακας 8: Query 4: επίδραση στρατηγικής join (2 exec $\times$ 1c/2GB).

Οι δύο χρόνοι σχεδόν ταυτίζονται, όπως αναμενόταν. Δεδομένου ότι οι 21 σταθμοί είναι αμελητέοι σε μέγεθος, ο μηχανισμός του join δεν έχει σημασία. Το κόστος εστιάζεται στα 63 εκατομμύρια ζεύγη και στο shuffle που ακολουθεί.

Συνολικά, και στα δύο ερωτήματα η μικρή πλευρά χωράει στη μνήμη, οπότε το broadcast είναι η καλύτερη επιλογή: η μικρή πλευρά εκπέμπεται παντού και αποφεύγεται πλήρως το ακριβό shuffle της μεγάλης. Το sort-merge και το shuffle-hash αξίζουν μόνο όταν είναι μεγάλες και οι δύο πλευρές, ενώ το shuffle-replicate-nl αποτελεί την τελευταία λύση για non-equi join, και την ακριβότερη.

9 Συμπεράσματα

Το πιο σημαντικό εύρημα αφορά τη μορφή αποθήκευσης. Απλώς μετατρέποντας τα δεδομένα σε Parquet επιτάχυνε το Query 2 σχεδόν εννέα φορές, χωρίς καμία άλλη αλλαγή. Ως προς τα API του Spark, δεν υπάρχει μία καθολικά σωστή επιλογή. Στο Query 1 ήταν καλύτερη η RDD υλοποίηση, στο join του Query 3 ήταν ταχύτερο το DataFrame, ενώ SQL και DataFrame είναι ισοδύναμα.

Ένα ακόμη σημαντικό συμπέρασμα είναι ότι η αύξηση των πόρων δεν συνεπάγεται απαραίτητα αύξηση στην απόδοση. Ένα ερώτημα που περιορίζεται από το shuffle, όπως το Query 4, παύει κάποια στιγμή να επιταχύνεται και επιδεινώνεται όταν οι ίδιοι πόροι κατανέμονται σε πολλούς μικρούς executors.